ForkOPD: Trust, but Rectify Privileged Teachers with Randomized Verified Forks

Anonymous ACL submission

Abstract

Reinforcement learning with verifiable rewards scores a whole reasoning trace with one bit, so per-step credit needs an estimate of how each step changed the chance of success. On-policy self-distillation reads that estimate off a privileged teacher: the same policy shown a reference solution. Measured against Monte Carlo values on Qwen3.5-9B, this hindsight signal is informative (Spearman 0.62), fades during training (0.38 after 400 steps), and errs most on near-miss prefixes that copy the reference and then fail. We therefore treat the teacher as a predictor and correct it. Following prediction-powered inference, ForkOPD (fork-rectified on-policy distillation) combines the teacher's prediction with a few verified continuations at randomly chosen segment boundaries. Its step values are conditionally unbiased for any predictor, as is the unclipped gradient estimate at the sampling policy; its trust coefficient and fork probabilities have closed forms; and it estimates online the share of predictor-dependent variance that hindsight removes. At a matched generation budget, ForkOPD improves the five-benchmark average by 3.0–4.3 points over DAPO on three base models and 2.0–2.8 over the strongest baseline (ten on Qwen3.5-9B, six on the others). With half of the references corrupted it loses 0.9 points, while methods that distill from or reweight with the uncorrected teacher lose 4.8–14.2.

1 Introduction

Reinforcement learning with verifiable rewards (RLVR) drives most recent progress in mathematical and programming reasoning (DeepSeek-AI, 2025; Yu et al., 2025). A response can run for thousands of tokens, and the verifier returns one bit at the end. GRPO (Shao et al., 2024) and its variants spread that bit evenly over the response, so a careless sign error and the idea that solved the problem receive the same update. Finer credit needs the value of each intermediate state: the probability that the policy succeeds from that prefix.

Two families estimate this value. Resampling methods restart from a prefix and average verified continuations (Kazemnejad et al., 2025), or spend the same budget at chosen branch points (Hou et al., 2025; Samanta et al., 2026). Their estimates are unbiased, and each one costs a rollout. Reading methods perform on-policy self-distillation: they condition the policy on a reference solution to obtain a privileged teacher and use the teacher's per-token log-probability shift on the student's own samples as dense credit, at the price of one forward pass (Zhao et al., 2026a; Hübotter et al., 2026). Later variants let the verifier set the sign and use the teacher for magnitude (Yang et al., 2026; Li et al., 2026; Huang et al., 2026).

Reading methods assume that the privileged score tracks the value it replaces. Against Monte Carlo values, a calibrated hindsight predictor reaches Spearman 0.62 on the initial policy and 0.38 after 400 steps of RL. Its largest errors sit on near misses (Figure 1): prefixes that follow the reference and already contain a mistake the policy tends to repeat. Hindsight keeps predicting success there and blames the last segment for 63% of failures (oracle: 18%), and gating by the verifier fixes the sign of a response but not this misplacement.

We therefore use hindsight as a predictor of value and correct it where it errs, following prediction-powered inference (Angelopoulos et al., 2023a; Zrnic and Candès, 2024), where a few gold labels placed with known probabilities remove the bias of many cheap predictions. In ForkOPD, short for fork-rectified on-policy distillation (Figure 2), the privileged teacher predicts the value at every segment boundary, and a random subset of boundaries, favoring uncertain predictions, receives a few verified continuations without the reference. Each rectified value is the prediction, anchored at the group baseline and scaled by a trust coefficient, plus an

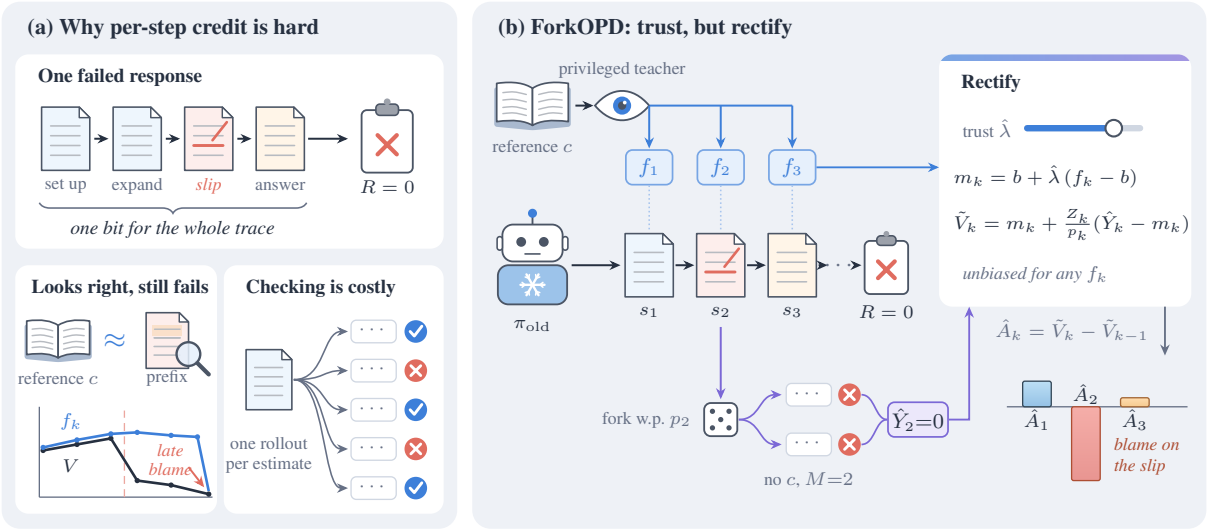


Figure 1: **Why per-step credit is hard, and how ForkOPD assigns it.** (a) The verifier returns one bit for the whole response. A prefix that resembles the reference c can already be lost, so the hindsight prediction f_k stays high past the slip and the blame arrives late; checking every prefix with rollouts costs one rollout per estimate. (b) ForkOPD predicts f_k at every boundary with one pass of the privileged teacher, forks a few boundaries at random (s_2 here, with $M{=}2$ continuations sampled without c), and rectifies the predictions with the verified outcomes, so the blame lands on the slip.

inverse-probability-weighted correction, and segment credit is the difference of consecutive values. These values are conditionally unbiased for any predictor, so a stale or misleading teacher adds variance but no bias, and the unclipped policy-gradient estimate is unbiased at the sampling policy. The trust coefficient in $[0, 1]$ and the fork probabilities have closed forms, and ForkOPD estimates from its own fork labels the share of predictor-dependent variance that hindsight removes.

We train Qwen3.5-4B, Qwen3.5-9B (Qwen Team, 2026) and Olmo-3-7B (Team Olmo, 2025) from base checkpoints at a matched generation budget, against ten baselines on the 9B model and six on the others. ForkOPD raises the five-benchmark average by 3.6, 4.3 and 3.0 points over DAPO and by 2.4, 2.8 and 2.0 over the strongest baseline, HSD. It reaches DAPO's final accuracy with 43% of the tokens and keeps Pass@256 on AIME 2026 5.1 points above the base model, where DAPO falls 3.1 below it. Our contributions are:

- A measurement of privileged hindsight as a value signal: informative, fading with training, and wrong on near misses (Section 3.2).
- ForkOPD, a credit estimator with conditionally unbiased step values for any predictor, closed-form trust and forks, and an online estimate of the variance hindsight removes (Sections 3.3–3.6).
- Matched-budget experiments against ten baselines on Qwen3.5-9B and six on two other models, including gradient quality against oracle advantages and wrong references (Section 4).

2 Related Work

Credit assignment in RLVR. Group-relative methods (Shao et al., 2024; Ahmadian et al., 2024; Yu et al., 2025) assign one outcome to every token, and shrinkage across prompts lowers the variance of their baselines (Zeng et al., 2026). VinePPO (Kazemnejad et al., 2025) replaced learned value networks, which rank reasoning steps poorly, with Monte Carlo continuations. Tree methods branch at high-entropy tokens (Hou et al., 2025), at steps with large attention mass (Liu et al., 2026), at semantically divergent points (Zhao et al., 2026b), or where expected gradient information is largest (Khomich et al., 2026), and reset methods resample from a self-localized error (Samanta et al., 2026). ForkOPD places its continuations at random with known probabilities and needs fewer of them because a predictor carries part of the estimate. Process reward models (Lightman et al., 2024; Wang et al., 2024; Setlur et al., 2025; Yuan et al., 2025) could serve as that predictor.

Privileged self-distillation. On-policy distillation (Agarwal et al., 2024) became self-distillation once the teacher was the student conditioned on privileged context (Zhao et al., 2026a; Hübotter

et al., 2026), which can suppress exploratory reasoning (Kim et al., 2026). Later methods let the verifier set the direction (Yang et al., 2026), condition the teacher on successful peers (Li et al., 2026), contrast correct and incorrect teachers (Heakl et al., 2026), or fold the gated signal into a trajectory-balance target (Huang et al., 2026). OC-GRPO puts the privileged context into the rollout and corrects with importance weights (Agrawal et al., 2026). ForkOPD keeps the reference out of every rollout and out of the update, and uses it only to predict values.

Prediction-powered and active inference. Prediction-powered inference (Angelopoulos et al., 2023a) gives valid estimates from many predictions and few labels. PPI++ (Angelopoulos et al., 2023b) adds the power-tuning coefficient that becomes our λ, and active inference (Zrnic and Candès, 2024) chooses which points to label using inverse-probability weights (Horvitz and Thompson, 1952) and Neyman-style allocation (Neyman, 1934). Unlike baselines as control variates (Greensmith et al., 2004), ForkOPD estimates intermediate values with no observed outcome and creates their labels with forks; unlike asymmetric critics (Vapnik and Vashist, 2009; Pinto et al., 2018), its privileged critic is the policy itself.

3 Methodology

3.1 Preliminaries

A policy π_θ answers a prompt x with a response y, and a verifier returns $R(x, y) \in \{0, 1\}$. We split y into segments $a_1, \ldots, a_K$ and write $s_k = (x, a_1, \ldots, a_k)$, with $s_0 = x$. A boundary is placed at the first double newline at least 64 tokens after the previous boundary, or after 1,024 tokens without one. Each decision depends only on the prefix, so boundaries are stopping times. With π_{old} the policy that generated the batch, the value of a state is its probability of eventual success,

$$V(s_k) = \mathbb{E}_{y' \sim \pi_{\text{old}}(\cdot|s_k)}\big[R(x, s_k \oplus y')\big], \quad (1)$$

and $V(s_K) = R(x, y)$. Transitions are deterministic, so segment a_k has advantage $A_k = V(s_k) - V(s_{k-1})$ and $\nabla_\theta J = \mathbb{E}\big[\sum_k A_k \nabla_\theta \log \pi_\theta(a_k \mid s_{k-1})\big]$. Each prompt is answered N times, and b is the leave-one-out mean reward of the other $N - 1$ responses (Ahmadian et al., 2024).

A training prompt often comes with a reference solution c that the deployed model never sees. Scoring the sampled tokens with π_{old} conditioned on c gives the hindsight evidence

$$L_k = \log \pi_{\text{old}}(a_{1:k} \mid x, c) - \log \pi_{\text{old}}(a_{1:k} \mid x), \quad (2)$$

the summed per-token log ratio ℓ_t up to boundary k. A prefix that agrees with c has high L_k, and reading methods turn ℓ_t into credit.

3.2 Motivation: how far can hindsight be trusted?

We freeze Qwen3.5-9B checkpoints from a DAPO run at steps 0, 200 and 400, sample 2,000 responses to held-out DeepMath prompts from each, and estimate every boundary's value with 64 verified continuations. The hindsight predictor is a logistic model on the features of Section 3.3, fit to oracle values on separate responses; a control omits c.

The signal is informative and fades. At step 0 the hindsight predictor reaches Spearman $\rho = 0.62$ with oracle values and the control 0.29. The correlation falls to 0.49 at step 200 and 0.38 at step 400, and the removable variance share η (Theorem 2) falls from 0.36 to 0.13 (Figure 3b). As RL sharpens the policy, ℓ_t shrinks on good and bad prefixes alike.

The error sits on near misses. On prefixes with $V(s_k) < 0.2$, the control overestimates by 0.06 to 0.07 at every level of agreement with the reference, the usual regression to the mean (Figure 3a). Hindsight overestimates by 0.03 on prefixes that disagree with the reference and by 0.27 on the top tercile of agreement: prefixes that copied the reference and already contain a mistake the policy tends to repeat.

Blame lands late, and gating keeps it there. Credit $f_k - f_{k-1}$, with endpoints b and R, puts the most negative segment of a failed response at its final segment in 63% of failures, against 18% under oracle advantages. Verifier gating gives a failed response negative total credit but still distributes it by hindsight: the rate is 41% for RLSD-style gating and 33% for HSD's peer-conditioned teacher.

3.3 A hindsight value predictor

For each boundary we compute z_k from one privileged forward pass and from the old-policy pass that DAPO already runs: the per-token evidence

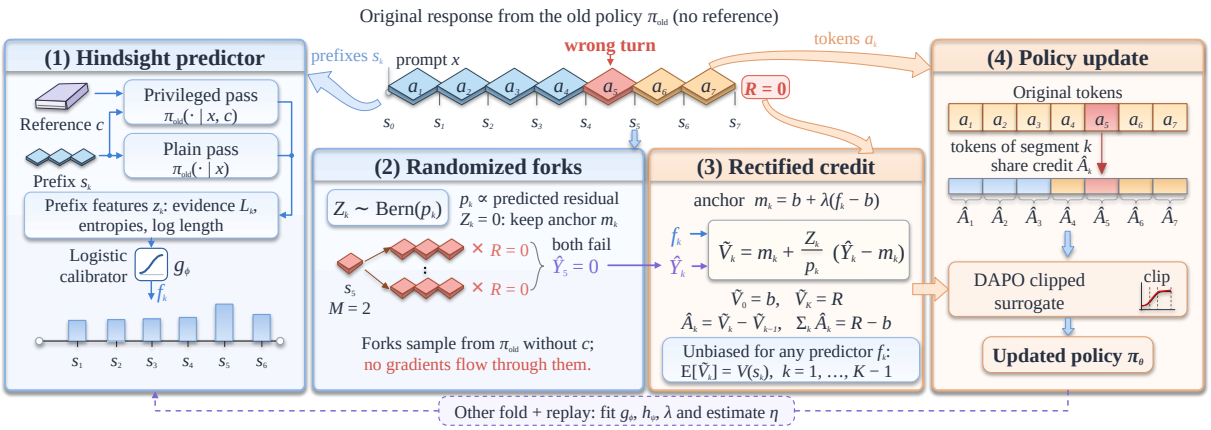


Figure 2: **One iteration of ForkOPD.** ① A privileged and a plain pass over each prefix give features z_k, and a logistic calibrator g_ϕ maps them to the prediction f_k. ② Each boundary is forked independently with probability p_k. At s_5, both continuations sampled from π_{old} without c fail, so $\hat{Y}_5{=}0$. ③ The rectified values $\tilde{V}_k$ correct the anchor m_k where a fork was taken, and their differences give the segment credit $\hat{A}_k$. ④ Every token of segment k receives $\hat{A}_k$ in the DAPO update. Bottom: each prompt fold fits g_ϕ, h_ψ and $\hat{\lambda}$, and estimates $\hat{\eta}$, for the other fold.

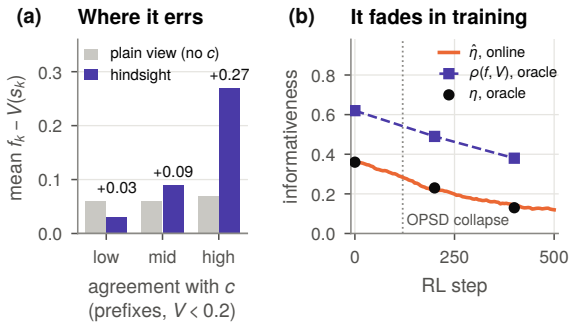


Figure 3: **Hindsight against oracle values** (Qwen3.5-9B; 64 continuations per boundary). **(a)** Error of two calibrated predictors on low-value prefixes by agreement with the reference. **(b)** Spearman correlation and η (Theorem 2) across a DAPO run, with ForkOPD's online estimate $\hat{\eta}$ of $\eta_c \le \eta$.

L_k/t_k, with t_k the response tokens in s_k; its increment $L_k - L_{k-1}$; the mean plain-view entropy over a_k; the privileged-minus-plain entropy gap over a_k; and $\log t_k$. A six-parameter logistic model maps z_k to $f_k = g_\phi(z_k) \in (0, 1)$. It is fit on fork labels only, with cross-fitting: the prompts of a batch are split into two folds, and each fold is predicted by a calibrator fit on the other fold plus a replay buffer of the previous 20 batches. This keeps f_k independent of the labels it is compared with, as Lemma 1 requires.

3.4 Randomized forks

At each interior boundary k we draw $Z_k \sim \text{Bernoulli}(p_k)$ independently. If $Z_k = 1$, we sample M continuations from s_k with π_{old} and without c, verify them, and record their mean $\hat{Y}_k$. As in VinePPO, the continuations receive no gradient. Proposition 1 shows that the best p_k grows with the expected squared error of the anchored prediction m_k defined below, so a linear model h_ψ, cross-fitted like g_ϕ, predicts that error beyond binomial noise from z_k. With a per-response budget B and a floor $p_{\min}$,

$$p_k = \min\{1,\ \max\{p_{\min},\ B\,u_k/\textstyle\sum_j u_j\}\},$$
$$u_k = \sqrt{f_k(1-f_k)/M + h_\psi(z_k)}. \tag{3}$$

The floor bounds every importance weight by $1/p_{\min}$. Equation (3) clips without renormalizing, so B is nominal and the p_k need not sum to it (Appendix A); budgets count realized forks. We use $B = 1$, $M = 2$ and $p_{\min} = 0.025$, which gives 1.06 forks per response.

3.5 Rectified values and segment credit

The prediction enters through an anchor $m_k = b + \lambda(f_k - b)$ that shrinks it toward the group baseline by a trust coefficient $\lambda \in [0, 1]$. The rectified value at an interior boundary is

$$\tilde{V}_k = m_k + \frac{Z_k}{p_k}\big(\hat{Y}_k - m_k\big), \tag{4}$$

with $\tilde{V}_0 = b$ and $\tilde{V}_K = R(x, y)$. An unforked boundary keeps the anchor. At a forked one the anchor cancels in expectation and the verified mean takes over. Segment credit is

$$\hat{A}_k = \tilde{V}_k - \tilde{V}_{k-1}, \qquad \sum_{k=1}^{K} \hat{A}_k = R(x, y) - b, \tag{5}$$

and every token of segment k receives advantage $\hat{A}_k$ in DAPO's clipped surrogate. Segment credits sum to the RLOO advantage, and hindsight and forks decide how it is split among segments; summed over tokens, each $\hat{A}_k$ is weighted by the length $|a_k|$ (Appendix B).

Trust coefficient. With $\omega_k = (1 - p_k)/p_k$, the summed variance of Equation (4) over a batch is minimized over $\lambda \in \mathbb{R}$ by a weighted regression slope $\lambda_{\mathrm{u}}^\star$ of $V_k - b$ on $f_k - b$, and over $[0, 1]$ by $\lambda_{\mathrm{c}}^\star = \mathrm{clip}(\lambda_{\mathrm{u}}^\star, 0, 1)$ (Proposition 1). We estimate the slope on the other fold,

$$\hat{\lambda} = \frac{\sum \omega_k \, (f_k - b) \, \frac{Z_k}{p_k} (\hat{Y}_k - b)}{\sum \omega_k \, (f_k - b)^2}, \qquad (6)$$

and clip it to $[0, 1]$. Centering at b makes λ a slope: a calibrated predictor that only knows the base rate gets $\lambda \approx 0$, whereas the uncentered form $m_k = \lambda f_k$ would give it $\lambda \approx 1$. We also log $\hat{\eta}$, an estimate of the variance share η_{c} removed by the clipped coefficient (Theorem 2; Appendix A).

Cost. Scoring adds one forward pass with c in context. Forks start partway through a response, so with $N = 8$ a prompt costs 15.6 response-equivalents of generation, close to DAPO's 16 (Algorithm 1, Appendix B).

3.6 Guarantees

Let $\mathcal{G}$ contain the response, c, the other responses of the group, and all labels from the other fold and from earlier batches. By construction, f_k, p_k, $\hat{\lambda}$ and b are $\mathcal{G}$-measurable. Proofs are in Appendix A.

Lemma 1 (Unbiased rectified value)**.** *If $p_k > 0$ and the continuations at s_k are drawn from $\pi_{\mathrm{old}}(\cdot \mid s_k)$ independently of $\mathcal{G}$ and Z_k, then $\mathbb{E}[\tilde{V}_k \mid \mathcal{G}] = V(s_k)$ for every $\mathcal{G}$-measurable f_k, λ and b.*

Theorem 1 (Unbiased policy gradient)**.** *Under the conditions of Lemma 1, if boundaries are stopping times, then $\hat{g} = \sum_k \hat{A}_k \nabla_\theta \log \pi_\theta(a_k \mid s_{k-1})$ satisfies $\mathbb{E}[\hat{g}] = \nabla_\theta J(\theta_{\mathrm{old}})$ at $\theta = \theta_{\mathrm{old}}$.*

The lemma places no condition on f_k: a miscalibrated, stale or adversarial predictor changes the variance of $\tilde{V}_k$ and leaves its conditional mean alone. The theorem needs causal boundaries, because a rule that merges segments after seeing the response makes boundaries depend on later tokens, which correlate with the outcome.

Proposition 1 (Variance and optimal design)**.** *With $V_k = V(s_k)$ and $\sigma_k^2 = V_k(1 - V_k)/M$,*

$$\mathrm{Var}(\tilde{V}_k \mid \mathcal{G}) = \frac{\sigma_k^2}{p_k} + \omega_k \, (V_k - m_k)^2. \qquad (7)$$

Let A, C and D be the ω-weighted sums over k of $(f_k - b)^2$, $(f_k - b)(V_k - b)$ and $(V_k - b)^2$. For fixed $\{p_k\}$, the summed variance is a term free of λ plus the predictor-dependent component $S(\lambda) = D - 2\lambda C + \lambda^2 A$. If $A > 0$, S is minimized over $\mathbb{R}$ by $\lambda_{\mathrm{u}}^\star = C/A$ and over $[0, 1]$ by $\lambda_{\mathrm{c}}^\star = \mathrm{clip}(C/A, 0, 1)$; if $A = 0$, $S \equiv D$ and we take $\lambda_{\mathrm{c}}^\star = 0$. The numerator of Equation (6) is unbiased for C. For fixed anchors and an exact budget B, the summed variance is minimized by $p_k^\star \propto \sqrt{\sigma_k^2 + (V_k - m_k)^2}$, truncated at one.

Theorem 2 (Variance saved by hindsight)**.** *For $A, D > 0$, $S(\lambda_{\mathrm{u}}^\star) = (1 - \eta) \, S(0)$ and $S(\lambda_{\mathrm{c}}^\star) = (1 - \eta_{\mathrm{c}}) \, S(0)$ with $\eta = C^2/(AD)$ and $\eta_{\mathrm{c}} = (2\lambda_{\mathrm{c}}^\star C - (\lambda_{\mathrm{c}}^\star)^2 A)/D$. Also, $0 \le \eta_{\mathrm{c}} \le \eta \le 1$, with $\eta_{\mathrm{c}} = \eta$ exactly when $\lambda_{\mathrm{u}}^\star \in [0, 1]$. For $A = 0$ or $D = 0$, nothing can be removed and we set $\eta = \eta_{\mathrm{c}} = 0$.*

At $\lambda = 0$, ForkOPD reduces to inverse-probability weighting with the same forks around the group baseline, so at $\lambda_{\mathrm{c}}^\star$ its variance never exceeds that of forks alone. η is a squared weighted correlation around the baseline and measures information rather than calibration; an anticorrelated predictor has $\eta > 0$ but $\eta_{\mathrm{c}} = 0$ (Appendix A). When $\lambda_{\mathrm{u}}^\star \in [0, 1]$, the cross-fitted $\hat{\lambda}$ gives $S(\hat{\lambda}) - S(\lambda_{\mathrm{c}}^\star)$ of order $1/n$ in the number of fork labels per fold (Angelopoulos et al., 2023b).

4 Experiments

4.1 Experimental Setup

4.1.1 Benchmarks and Baselines

Models and data. We train from base checkpoints so that every gain comes from RL: Qwen3.5-4B-Base and Qwen3.5-9B-Base (Qwen Team, 2026), hybrids of Gated DeltaNet and attention layers, and Olmo-3-7B (Team Olmo, 2025), whose open pretraining data can be checked for contamination. For mathematics we filter DeepMath-103K (He et al., 2026) to problems of difficulty six or above whose base-model pass rate over eight samples is strictly between 0 and 1, which leaves 24.3K prompts with three R1 solutions each, one of which we use as c. For programming we use the 12.1K human-seeded problems of rStar-Coder (Liu et al., 2025), with oracle solutions as c and verified tests.

Evaluation. Mathematics: AIME 2026 and HMMT February 2026 (released after or within weeks of the base models), BeyondAIME, IMO-AnswerBench (Luong et al., 2025) and AIME 2025. Programming: LiveCodeBench v6 (Jain et al., 2025) and LCB-2026, 187 AtCoder and Codeforces problems from January to August 2026 collected with the LiveCodeBench pipeline. Out of domain: GPQA-Diamond (Rein et al., 2024). We report avg@16 on AIME and HMMT and avg@8 elsewhere, sampling at temperature 1.0 and top-p 0.95 with an 8,192-token budget.

Baselines and budget. We group ten baselines, all run on Qwen3.5-9B and six on the other models (Table 2), by how they obtain credit: outcome only (GRPO, DAPO; Shao et al., 2024; Yu et al., 2025), resampling (VinePPO, TreeRL, SRPO), privileged self-distillation (OPSD, RLSD, HSD), privileged rollouts with importance correction (OC-GRPO) and distributional (FlowBalance), with citations in Section 2. All run in verl (Sheng et al., 2025) on a shared DAPO backbone with the same data, verifier, length cap and optimizer, with their own coefficients tuned on a held-out split (Appendix D). We match generation per step, counting every continuation: methods without forks use $N = 16$, ForkOPD uses $N = 8$ with $B = 1$ and $M = 2$, and VinePPO, TreeRL and SRPO use $N = 8$ plus their own continuations. A 9B run generates about 3.9×10^9 tokens in 500 steps (Appendix C).

4.1.2 Implementation Details

We train Qwen3.5-9B for 500 steps (fewer for the other models) with 128 prompts per step, learning rate 10^{-6}, DAPO clipping $(0.2, 0.28)$, no KL penalty and responses up to 8,192 tokens, on 64 H100 GPUs. Qwen3.5-9B responses have 13.2 segments on average. Results are means over three seeds (Appendix E); Appendix B lists all settings.

4.2 Main Results

ForkOPD is best on all five benchmarks (Table 1) and raises the 9B average by 4.28 points over DAPO, 2.76 over HSD and 3.16 over SRPO. A paired permutation test against HSD on the 590 pooled problems gives $p = 0.003$.

The baselines sort by how much they trust the privileged view. OPSD distills toward it and ends 6.30 points below DAPO, its mean response length falling from 3.9K to 1.6K tokens (Kim et al., 2026). RLSD matches DAPO and FlowBalance gains 0.74. SRPO and HSD, which obtain intermediate outcomes through resets and successful peers, are the strongest baselines. VinePPO trails DAPO by 0.72 points: at a matched budget, forking at every boundary leaves it 36 prompts per step.

On Qwen3.5-4B and Olmo-3-7B (six baselines, Table 2), ForkOPD again leads, by 2.40 and 2.02 points over HSD. Olmo starts lower, so fewer of its prompts produce mixed outcomes and there is less credit to redistribute. On programming, where c is an oracle program and the verifier runs unit tests, ForkOPD improves LiveCodeBench v6 by 2.5 points and the post-release LCB-2026 split by 2.2 over HSD. On GPQA-Diamond, which no method trains on, the math-trained checkpoints lie within 2.6 points of each other and ForkOPD is highest at 44.1.

Efficiency. ForkOPD reaches DAPO's final 9B average after 43% of the token budget, against 74% for HSD and 81% for SRPO (Figure 4a), and after 55% of DAPO's wall-clock time (Figure 4b). A full run takes 1.27 times as long as DAPO, because mid-response forks batch poorly and the privileged pass adds compute.

4.3 Ablations

Rectification. Hindsight alone ($\lambda = 1$, no forks) is 0.30 points above DAPO, forks alone ($\lambda = 0$) 1.30, and the combination 3.64. Beyond the variance saving, clipping explains part of the gap: with forks alone the importance-weighted corrections carry the whole learning signal, and 27% of their magnitude falls on tokens whose probability ratio is already clipped, against 9% with the anchor. Centering matters too: the uncentered anchor $m_k = \lambda f_k$ loses 0.74. Fixed $\lambda = 1$ loses 0.44 and $\lambda = 0.5$ loses 0.94; the calibrator is fit on slightly stale labels, and $\hat{\lambda}$ follows its drifting slope.

Fork placement. Placing forks by predicted residual beats uniform placement by 1.14 points and entropy-based branching by 0.84. With $p_{\min} = 0.025$ the median importance weight on forked boundaries is 6.8 and the 99th percentile 38. Without the floor, weights reached 1.4×10^3 and one seed diverged.

Predictor and reference. A predictor without c (plain-view features only) loses 1.84 points, so most of the predictor's value comes from the privileged view. Measuring evidence against π_{ref} instead of π_{old}, as FlowBalance does, loses 1.54 and

Family	Method	AIME26	HMMT26	BeyondAIME	IMO-AB	AIME25	Avg.	Δ
	Base model	12.1	6.3	5.2	14.8	13.5	10.38	–
Outcome only	GRPO	31.5	17.2	13.6	27.4	30.9	24.12	−1.66
	DAPO	33.8	18.6	14.9	28.9	32.7	25.78	0.00
Resampling	VinePPO	32.6	18.1	14.2	28.6	31.8	25.06	−0.72
	TreeRL	33.2	18.0	14.4	28.3	32.1	25.20	−0.58
	SRPO	35.0	19.6	15.8	30.0	34.1	26.90	+1.12
Self-distillation	OPSD	26.4	12.9	9.8	22.6	25.7	19.48	−6.30
	RLSD	33.6	18.9	14.6	29.4	32.8	25.86	+0.08
	HSD	35.4	20.1	15.9	30.6	34.5	27.30	+1.52
Privileged rollouts	OC-GRPO	34.2	19.0	15.1	29.7	33.4	26.28	+0.50
Distributional	FlowBalance	34.6	19.3	15.2	29.9	33.6	26.52	+0.74
Rectified (ours)	ForkOPD	**38.9**	**22.7**	**18.1**	**33.2**	**37.4**	**30.06**	+4.28

Table 1: **Mathematical reasoning on Qwen3.5-9B** trained from the base checkpoint with about 3.9×10^9 generated tokens per method. avg@16 for AIME and HMMT, avg@8 otherwise; mean of three seeds. Best in **bold**, second underlined; Δ is the change in average relative to DAPO.

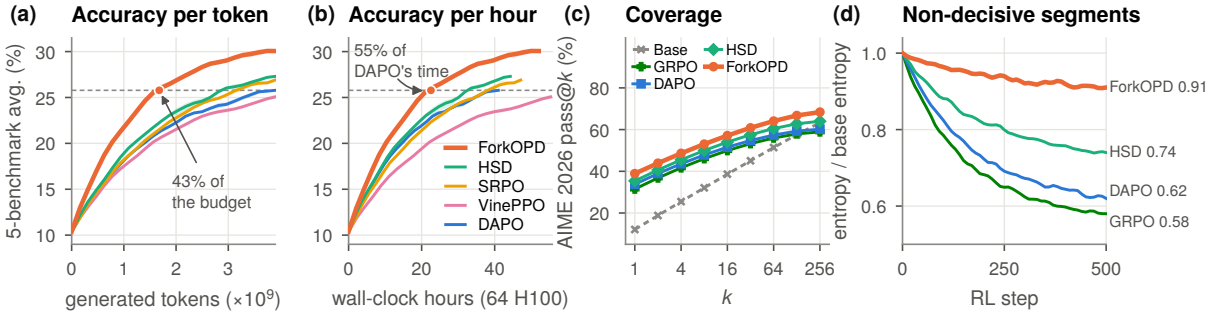


Figure 4: **Training dynamics on Qwen3.5-9B. (a, b)** Five-benchmark average against generated tokens and wall-clock hours on 64 H100 GPUs; bands span three seeds, the dashed line marks DAPO's final average. **(c)** Pass@k on AIME 2026 after training. **(d)** Token entropy on non-decisive segments ($|A_k| < 0.02$ under oracle values), relative to the base model.

	Math avg.		Qwen3.5-9B		
Method	Qw-4B	Olmo	LCB v6	LCB-26	GPQA
Base model	5.64	3.64	18.4	9.6	33.1
GRPO	17.42	11.90	34.2	19.8	41.5
DAPO	18.60	12.96	35.9	21.0	42.2
SRPO	19.56	13.72	36.8	21.9	42.6
RLSD	18.60	12.82	35.5	20.6	41.9
HSD	19.84	13.98	37.1	22.3	42.8
FlowBalance	19.20	13.34	36.4	21.5	42.4
ForkOPD	**22.24**	**16.00**	**39.6**	**24.5**	**44.1**

Table 2: **Other base models and domains** (six baselines). Left: five-benchmark math average. Right: Qwen3.5-9B on programming (avg@8 pass rate) and on GPQA-Diamond with the math-trained checkpoints.

lowers $\hat{\eta}$ to 0.15, since the log ratio then also measures drift from initialization. With only the final answer as c, $\hat{\eta}$ averages 0.13 and accuracy drops by 1.24; with all three R1 solutions, $\hat{\eta}$ rises from 0.23 to 0.29 and accuracy by 0.26.

Estimator and budget. Without cross-fitting, rectified values on failed responses are biased by +0.04 and accuracy drops by 0.84. Fixed 256-token windows lose 0.54 against newline segmentation. Non-causal segmentation, which merges the shortest adjacent segments to cap K at 32, loses 0.14 and voids Theorem 1. Doubling the fork budget gains 0.36 at 1.5 times the tokens, and halving it loses 0.94.

4.4 Analysis

Gradient quality. Hindsight credit without forks levels off at a cosine of 0.62 to 0.70 (Table 4), a bias that more data does not remove (Appendix A). Outcome-only and fork-based credit keep improving with data, and ForkOPD gives the best single-batch gradient. At step 200, $S(0)$ is 58% of the variance of forks alone, and hindsight can remove $\eta = 0.23$ of it, about 13% of the total; residual-

Component	Variant	Avg.	
Rectification	hindsight only (λ=1, B=0)	18.90	−3.34
	forks only (λ=0)	19.90	−2.34
	no anchor ($m_k = \lambda f_k$)	21.50	−0.74
	fixed λ=0.5	21.30	−0.94
	fixed λ=1	21.80	−0.44
Fork placement	uniform	21.10	−1.14
	highest entropy	21.40	−0.84
	self-localized error	21.60	−0.64
	no floor ($p_{\min}$=0)†	21.80	−0.44
Reference c	final answer only	21.00	−1.24
	three R1 solutions	22.50	+0.26
Predictor	plain view only (no c)	20.40	−1.84
	evidence features only	21.20	−1.04
	π_{ref} instead of π_{old}	20.70	−1.54
Estimator	no cross-fitting	21.40	−0.84
	fixed 256-token windows	21.70	−0.54
	non-causal segmentation	22.10	−0.14
Budget	B=0.5	21.30	−0.94
	B=2 (1.5× tokens)	22.60	+0.36
	M=1, B=2‡	21.70	−0.54
	M=4, B=0.5	22.30	+0.06

Table 3: **Ablations on Qwen3.5-4B** (five-benchmark average; full method 22.24, DAPO 18.60). Each row changes one component; gray: change from the full method. †One of three seeds diverged after step 300; mean of the other two. $^\ddagger h_\psi$ fit to total squared residuals, no $\hat{\eta}$ (App. B).

Credit source	cos@1	cos@8	Limit
Uniform (DAPO)	0.21	0.52	0.99
RLSD gating	0.23	0.47	0.62
HSD peer-conditioned	0.27	0.54	0.70
Hindsight only (λ=1, B=0)	0.28	0.53	0.66
Forks only (λ=0)	0.31	0.68	0.98
ForkOPD	**0.34**	**0.72**	0.99

Table 4: **Gradient quality** on Qwen3.5-9B at step 200: cosine between each method's gradient and the gradient with oracle advantages (32 continuations per boundary) on the same responses, over one or eight batches of 128 prompts. Limit: asymptote of a fit of $\cos^{-2}$ linear in $1/n$ for $n \in \{1, 2, 4, 8\}$; values below one indicate bias.

Method	q=0	q=0.2	q=0.5	Δ
OPSD	19.48	14.10	5.30	−14.18
RLSD	25.86	23.40	19.80	−6.06
FlowBalance	26.52	24.60	21.70	−4.82
OC-GRPO	26.28	25.60	24.60	−1.68
ForkOPD	30.06	29.70	29.20	**−0.86**
$\hat{\eta}$ of ForkOPD	0.24	0.18	0.10	–

Table 5: **Wrong references.** Five-benchmark average on Qwen3.5-9B when a fraction q of references is replaced by fluent solutions to other problems on the same topic. Last row: mean $\hat{\eta}$ of ForkOPD over training.

based allocation adds the rest.

When the reference is wrong. By Lemma 1, a bad predictor adds variance but no bias to $\tilde{V}_k$. We replace the reference of a fraction q of training prompts with a fluent solution to another problem on the same topic, keeping the correct answer in the verifier (Table 5). At $q = 0.5$, OPSD falls below the base model, RLSD loses 6.06 points, FlowBalance 4.82 and OC-GRPO 1.68. ForkOPD loses 0.86 while its mean $\hat{\eta}$ drops from 0.24 to 0.10, and it still exceeds HSD and DAPO, which do not read c.

Coverage. RLVR often raises Pass@1 while lowering Pass@k for large k (Yue et al., 2025). At $k = 256$ on AIME 2026, DAPO reaches 60.2% against the base model's 63.3%, and ForkOPD 68.4% (Figure 4c). On segments with near-zero oracle advantage, DAPO's token entropy falls to 0.62 of the base model's and ForkOPD's stays at 0.91 (Figure 4d): uniform credit sharpens every token of a rewarded response, while ForkOPD's mean absolute credit on these segments is 0.31 of DAPO's.

What $\hat{\eta}$ records. On Qwen3.5-9B, $\hat{\eta}$ falls from 0.37 to 0.22 by step 200 and 0.14 by step 400, tracking the oracle η without oracle rollouts (Figure 3b), while $\hat{\lambda}$ stays between 0.81 and 0.94: refitting keeps the predictor calibrated as its information declines. OPSD's response length starts to collapse at step 120, when $\hat{\eta}$ is still 0.29, so the privileged view was still informative and the distillation objective is the likely cause.

5 Conclusion

Privileged hindsight carries real information about which reasoning steps matter, less of it as training proceeds, and least on the near misses where credit matters most. ForkOPD corrects the signal with about one verified fork per response, which removes its bias from the step values, keeps most of its variance reduction, and yields $\hat{\eta}$ as a running measure of what the privileged view knows. The construction carries over to other cheap value predictors, such as process reward models.

Limitations

ForkOPD needs a reference solution for each training prompt. Many math and code datasets provide one and other domains often do not; the answer-only variant in Table 3 shows the cost of a weaker reference. Forks restart from re-fed text, which reproduces the state of attention layers up to numerical noise and needs recurrent-state snapshots for layers such as Gated DeltaNet. The variance formulas assume binary rewards; graded rewards change the noise term σ_k^2 but not the estimator. Our guarantees concern the unclipped estimator $\hat{g}$ at θ_{old}. Training uses DAPO's clipped, token-normalized surrogate, whose gradient matches $\hat{g}$ only at θ_{old} and up to the normalizer, so we do not claim that the teacher cannot shift the expected update of clipped training; the ablations show that clipping interacts with the importance-weighted corrections. We study mathematics and competitive programming with models up to 9B parameters; agentic tasks with long horizons, where restarting from an intermediate state is expensive or impossible, remain open.

References

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. 2024. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*.

Priyank Agrawal, Ankur Samanta, Shervin Ghasemlou, Boris Vidolov, Jalaj Bhandari, Kavosh Asadi, Daniel Jiang, and Aditya Modi. 2026. Off-context GRPO: Learning to reason on hard problems using privileged information. *arXiv preprint arXiv:2607.19313*.

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. 2024. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Anastasios N. Angelopoulos, Stephen Bates, Clara Fannjiang, Michael I. Jordan, and Tijana Zrnic. 2023a. Prediction-powered inference. *Science*, 382(6671):669–674.

Anastasios N. Angelopoulos, John C. Duchi, and Tijana Zrnic. 2023b. PPI++: Efficient prediction-powered inference. *arXiv preprint arXiv:2311.01453*.

DeepSeek-AI. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645:633–638.

Evan Greensmith, Peter L. Bartlett, and Jonathan Baxter. 2004. Variance reduction techniques for gradient estimates in reinforcement learning. *Journal of Machine Learning Research*, 5:1471–1530.

Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. 2026. DeepMath-103K: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning. In *International Conference on Learning Representations*.

Ahmed Heakl, Abdelrahman M. Shaker, Youssef Mohamed, Rania Elbadry, Omar Fetouh, Fahad Shahbaz Khan, and Salman Khan. 2026. CEPO: RLVR self-distillation using contrastive evidence policy optimization. *arXiv preprint arXiv:2605.19436*.

Daniel G. Horvitz and Donovan J. Thompson. 1952. A generalization of sampling without replacement from a finite universe. *Journal of the American Statistical Association*, 47(260):663–685.

Zhenyu Hou, Ziniu Hu, Yujiang Li, Rui Lu, Jie Tang, and Yuxiao Dong. 2025. TreeRL: LLM reinforcement learning with on-policy tree search. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Zixun Huang, Kishan Panaganti, Haitao Mi, and Leowei Liang. 2026. FlowBalance: Verifier-grounded self-improvement from on-policy reasoning experience. *arXiv preprint arXiv:2609.03241*.

Jonas Hübotter, Frederike Lübeck, Lejs Deen Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. 2026. Reinforcement learning via self-distillation. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 48677–48721.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*.

Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. 2025. VinePPO: Refining credit assignment in RL training of LLMs. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*.

Nikita Khomich, Leopold Hermansson, and Ido Hakimi. 2026. EPIG-Tree: Compute-optimal branching for gradient-efficient reinforcement learning. *arXiv preprint arXiv:2609.20004*.

Jeonghye Kim, Xufang Luo, Minbeom Kim, Sangmook Lee, Dohyung Kim, Jiwon Jeon, Dongsheng Li, and Yuqing Yang. 2026. Why does self-distillation (sometimes) degrade the reasoning capability of LLMs? *arXiv preprint arXiv:2603.24472*.

Yu Li, Shu Hong, and Tian Lan. 2026. Localizing credit at the divergence: Path-conditioned self-distillation for LLM reasoning. *arXiv preprint arXiv:2606.15576*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *International Conference on Learning Representations*.

Runze Liu, Jiakang Wang, Yuling Shi, Zhihui Xie, Chenxin An, Kaiyan Zhang, Jian Zhao, Xiaodong Gu, Lei Lin, Wenping Hu, Xiu Li, Fuzheng Zhang, Guorui Zhou, and Kun Gai. 2026. Attention as a compass: Efficient exploration for process-supervised RL in reasoning models. In *International Conference on Learning Representations*.

Yifei Liu, Li Lyna Zhang, Yi Zhu, Bingcheng Dong, Xudong Zhou, Ning Shang, Fan Yang, Cheng Li, and Mao Yang. 2025. rStar-Coder: Scaling competitive code reasoning with a large-scale verified dataset. In *Advances in Neural Information Processing Systems*.

Thang Luong, Dawsen Hwang, Hoang H. Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, Alex Zhai, Clara Huiyi Hu, Henryk Michalewski, Jimin Kim, Jeonghyun Ahn, Junhwi Bae, Xingyou Song, Trieu H. Trinh, Quoc V. Le, and Junehyuk Jung. 2025. Towards robust mathematical reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Jerzy Neyman. 1934. On the two different aspects of the representative method: The method of stratified sampling and the method of purposive selection. *Journal of the Royal Statistical Society*, 97(4):558–625.

Lerrel Pinto, Marcin Andrychowicz, Peter Welinder, Wojciech Zaremba, and Pieter Abbeel. 2018. Asymmetric actor critic for image-based robot learning. In *Robotics: Science and Systems*.

Qwen Team. 2026. Qwen3.5: Towards native multimodal agents. https://qwen.ai/blog?id=qwen3.5.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2024. GPQA: A graduate-level Google-proof Q&A benchmark. In *Conference on Language Modeling*.

Ankur Samanta, Akshayaa Magesh, Ayush Jain, Youliang Yu, Daniel Jiang, Kavosh Asadi, Kaveh Hassani, Paul Sajda, Jalaj Bhandari, and Yonathan Efroni. 2026. Credit assignment with resets in language model reasoning. *arXiv preprint arXiv:2605.25507*.

Amrith Setlur, Chirag Nagpal, Adam Fisch, Xinyang Geng, Jacob Eisenstein, Rishabh Agarwal, Alekh Agarwal, Jonathan Berant, and Aviral Kumar. 2025. Rewarding progress: Scaling automated process verifiers for LLM reasoning. In *International Conference on Learning Representations*.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. HybridFlow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*.

Team Olmo. 2025. Olmo 3. *arXiv preprint arXiv:2512.13961*.

Vladimir Vapnik and Akshay Vashist. 2009. A new learning paradigm: Learning using privileged information. *Neural Networks*, 22(5–6):544–557.

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. 2024. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Chenxu Yang, Chuanyu Qin, Qingyi Si, Minghui Chen, Naibin Gu, Dingyu Yao, Zheng Lin, Weiping Wang, Jiaqi Wang, and Nan Duan. 2026. Self-distilled RLVR. *arXiv preprint arXiv:2604.03128*.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Juncai Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, and 17 others. 2025. DAPO: An open-source LLM reinforcement learning system at scale. In *Advances in Neural Information Processing Systems*.

Lifan Yuan, Wendi Li, Huayu Chen, Ganqu Cui, Ning Ding, Kaiyan Zhang, Bowen Zhou, Zhiyuan Liu, and Hao Peng. 2025. Free process rewards without process labels. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. 2025. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*.

Guanning Zeng, Zhaoyi Zhou, Daman Arora, and Andrea Zanette. 2026. Shrinking the variance: Shrinkage baselines for reinforcement learning with verifiable rewards. In *Proceedings of the 43rd International Conference on Machine Learning*.

Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. 2026a. Self-distilled reasoner: On-policy self-distillation for large language models. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 162433–162448.

Ziqi Zhao, Zhaochun Ren, Jiahong Zou, Liu Yang, Zhiwei Xu, Xuri Ge, Zhumin Chen, Xinyu Ma, Daiting Shi, Shuaiqiang Wang, Dawei Yin, and Xin Xin. 2026b. Reinforced efficient reasoning via semantically diverse exploration. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 47994–48007.

Tijana Zrnic and Emmanuel J. Candès. 2024. Active statistical inference. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*.

A Proofs

Throughout, $\nabla_k = \nabla_\theta \log \pi_\theta(a_k \mid s_{k-1})$ at $\theta = \theta_{\text{old}}$, and $\Delta_k = \hat{Y}_k - m_k$.

Lemma 1. Given $\mathcal{G}$, the quantities f_k, λ, b, m_k and p_k are constants, Z_k is independent of the continuations, and $\mathbb{E}[\hat{Y}_k \mid \mathcal{G}] = V(s_k)$ because each continuation is an independent draw from $\pi_{\text{old}}(\cdot \mid s_k)$. Hence $\mathbb{E}[\tilde{V}_k \mid \mathcal{G}] = m_k + \frac{\mathbb{E}[Z_k]}{p_k}\big(V(s_k) - m_k\big) = V(s_k)$. □

Theorem 1. Because the boundary after a_k is a stopping time, $\mathbb{E}[R \mid s_k] = V(s_k)$ and ∇_k is a function of s_k. By Lemma 1 and the tower rule, $\mathbb{E}[\tilde{V}_k \nabla_k] = \mathbb{E}[V(s_k)\nabla_k] = \mathbb{E}[R\,\nabla_k]$ for $1 \le k < K$, and trivially for $k = K$. For the subtracted term, $\mathbb{E}[\tilde{V}_{k-1}\nabla_k] = \mathbb{E}[V(s_{k-1})\nabla_k] = 0$, since $V(s_{k-1})$ is a function of s_{k-1} and the score of a segment ending at a stopping time has conditional mean zero given s_{k-1}. For $k = 1$, $\tilde{V}_0 = b$ is independent of the response. Summing, $\mathbb{E}[\hat{g}] = \mathbb{E}\big[R \sum_k \nabla_k\big] = \mathbb{E}[R\,\nabla_\theta \log \pi_\theta(y \mid x)] = \nabla_\theta J(\theta_{\text{old}})$. Only the boundaries must be causal; f_k and p_k may depend on the entire response. □

Proposition 1. Conditional on $\mathcal{G}$, $\mathbb{E}[\Delta_k] = V_k - m_k$ and $\mathbb{E}[\Delta_k^2] = \sigma_k^2 + (V_k - m_k)^2$. Since $Z_k^2 = Z_k$, $\mathrm{Var}(\tilde{V}_k \mid \mathcal{G}) = \mathbb{E}[\Delta_k^2]/p_k - (\mathbb{E}\Delta_k)^2$, which gives Equation (7). Writing $V_k - m_k = (V_k - b) - \lambda(f_k - b)$ and expanding the square, the λ-dependent part of the summed variance is $S(\lambda) = D - 2\lambda C + \lambda^2 A$. If $A > 0$, S is a strictly convex quadratic with minimizer C/A over $\mathbb{R}$; it decreases to the left of C/A and increases to the right, so its minimizer over $[0, 1]$ is the projection $\mathrm{clip}(C/A, 0, 1)$. If $A = 0$, then $f_k = b$ wherever $\omega_k > 0$, so $C = 0$ and $S \equiv D$; every λ is optimal and we take 0. In Equation (6), $\mathbb{E}[Z_k(\hat{Y}_k - b)/p_k \mid \mathcal{G}] = V_k - b$, so the numerator is unbiased for C and the denominator is A, which is known. For the allocation, up to a term constant in p the summed variance is $\sum_k \mathbb{E}[\Delta_k^2]/p_k$; minimizing under $\sum_k p_k = B$ and $p_k \le 1$ with a Lagrange multiplier gives $p_k^\star = \min\{1, \tau\sqrt{\mathbb{E}[\Delta_k^2]}\}$, with τ chosen so that the constraint holds. □

Equation (3) versus $p_k^\star$. Equation (3) plugs u_k^2 in for $\mathbb{E}[\Delta_k^2]$, with $f_k(1 - f_k)/M$ in place of σ_k^2, and applies the cap and the floor after scaling by $B/\sum_j u_j$ without re-solving for the scale. Its probabilities therefore need not sum to B. For example, with $B = 1$, normalized weights $(0.99, 0.01)$ and $p_{\min} = 0.025$, it gives $(0.99, 0.025)$, which sums to 1.015. A response with $K - 1$ interior boundaries has $\sum_k p_k \ge (K - 1)p_{\min}$, so no allocation with the floor meets the budget when $(K - 1)p_{\min} > B$, that is, beyond 40 interior boundaries at our settings; Equation (3) then keeps the floor and spends more than B. An exact version would choose the scale τ by bisection so that $\sum_k \mathrm{clip}(\tau u_k, p_{\min}, 1) = B$, which is feasible when $(K - 1)p_{\min} \le B \le K - 1$. Lemma 1 and Theorem 1 require only $p_k > 0$ and $\mathcal{G}$-measurability, so neither choice affects unbiasedness, and we account for the realized number of forks (Appendix C).

Theorem 2. Here $S(0) = D$. From $S(\lambda) = D - 2\lambda C + \lambda^2 A$, $S(0) - S(\lambda) = 2\lambda C - \lambda^2 A$ for every λ; at $\lambda = \lambda_{\text{c}}^\star$ this gives η_{c}, and at $\lambda = \lambda_{\text{u}}^\star = C/A$ it gives $C^2/A = \eta D$. Completing the square, $S(\lambda) = S(\lambda_{\text{u}}^\star) + A(\lambda - \lambda_{\text{u}}^\star)^2$, so $S(0) - S(\lambda_{\text{c}}^\star) = \eta D - (C - \lambda_{\text{c}}^\star A)^2/A$ and

$$\eta_{\text{c}} = \eta - \frac{(C - \lambda_{\text{c}}^\star A)^2}{AD}.$$

Finally, $\eta \le 1$ by the Cauchy–Schwarz inequality for the weights ω_k. $\eta_{\text{c}} \ge 0$ because 0 is feasible and $\lambda_{\text{c}}^\star$ is optimal over $[0, 1]$, and $\eta_{\text{c}} \le \eta$ because $\lambda_{\text{u}}^\star$ is optimal over $\mathbb{R}$. The gap vanishes exactly when $\lambda_{\text{c}}^\star = C/A$, that is, when $0 \le C \le A$. If $C \le 0$, then $\lambda_{\text{c}}^\star = 0$ and $\eta_{\text{c}} = 0$; if $C > 0$, then

$S'(0) = -2C < 0$, so $S(\lambda_c^\star) < S(0)$ and $\eta_c > 0$. If $A = 0$, S is constant; if $D = 0$, then $C = 0$ by Cauchy–Schwarz, $\lambda_c^\star = 0$ and $S(\lambda_c^\star) = S(0) = 0$. In both cases nothing can be removed. □

Example: $\eta_c < \eta$. Take $b = 0.5$ and two boundaries with equal weights ω, $f = (0.2, 0.8)$ and $V = (0.8, 0.2)$. Then $A = D = 0.18\,\omega$ and $C = -0.18\,\omega$, so $\lambda_u^\star = -1$ and $\eta = 1$: with negative trust the predictor would remove all of $S(0)$. The feasible coefficient is $\lambda_c^\star = 0$, which leaves $S(\lambda_c^\star) = S(0)$, so $\eta_c = 0$ and ForkOPD falls back to forks alone. When $C > A$, the slope exceeds one, $\lambda_c^\star = 1$ and $\eta_c = (2C - A)/D < \eta$.

Estimating η_c. Let $\hat{C}$ be the numerator of Equation (6), unbiased for C, and A its denominator, so that $\hat{\lambda} = \mathrm{clip}(\hat{C}/A, 0, 1)$, with $\hat{\lambda} = 0$ if $A = 0$. For $M \geq 2$, $\hat{\sigma}_k^2 = \hat{Y}_k(1 - \hat{Y}_k)/(M - 1)$ is unbiased for σ_k^2, and $\hat{S}_0 = \sum_k \omega_k \frac{Z_k}{p_k}\left[(\hat{Y}_k - b)^2 - \hat{\sigma}_k^2\right]$ is unbiased for $S(0) = D$. We report the plug-in estimate of η_c,

$$\hat{\eta} = \left(2\hat{\lambda}\hat{C} - \hat{\lambda}^2 A\right)/\hat{S}_0,$$

computed on the other fold and clipped to $[0, 1]$, with $\hat{\eta} = 0$ when $A = 0$ or $\hat{S}_0 \leq 0$. Clipping is a projection onto $[0, 1]$ and hence non-expansive, so $|\hat{\lambda} - \lambda_c^\star|$ never exceeds the error of the unclipped slope estimate. When $0 \leq \hat{C} \leq A$ the clip on $\hat{\lambda}$ is inactive and $\hat{\eta} = \hat{\lambda}^2 A/\hat{S}_0 = \hat{C}^2/(A\hat{S}_0)$, the plug-in estimate of η; when $\hat{C} \leq 0$, $\hat{\eta} = 0$. As a ratio of estimates, $\hat{\eta}$ is not itself unbiased. With $M = 1$, $\hat{Y}_k \in \{0, 1\}$, and from a single Bernoulli draw only functions linear in V_k have unbiased estimators; neither $\sigma_k^2 = V_k(1 - V_k)$ nor $(V_k - b)^2$ does, so we do not report $\hat{\eta}$ for $M = 1$. The rectified values and $\hat{\lambda}$ need only $\mathbb{E}[\hat{Y}_k \mid \mathcal{G}] = V_k$ and are unaffected.

Gated credit. Methods such as RLSD and FlowBalance assign token t the credit $\mathrm{sgn}(R - b)\,\phi_t$, with $\phi_t \geq 0$ computed from the log ratios ℓ. Its expected update is $\sum_t \mathbb{E}[\mathrm{sgn}(R - b)\,\phi_t \nabla_t]$. The weight ϕ_t depends on c and, for trajectory-level scores, on tokens after t, so this direction is neither $\nabla_\theta J$ nor a per-state rescaling of it, and the deviation does not average out. Within a response, the only information it carries about which step mattered is the single outcome R. A fork after a step supplies an intermediate outcome, which is what Lemma 1 uses.

Algorithm 1 One iteration of ForkOPD

Require: prompts with references $\{(x, c)\}$, policy π_θ, nominal budget B, continuations per fork M

1: $\pi_{\mathrm{old}} \leftarrow \pi_\theta$; sample N responses per prompt; verify
2: Segment responses causally; run one privileged pass
3: Compute z_k; predict f_k with the other fold's g_ϕ
4: Set p_k by Eq. (3); draw $Z_k \sim \mathrm{Bern}(p_k)$
5: For each $Z_k = 1$: M continuations from s_k without c; $\hat{Y}_k \leftarrow$ mean reward
6: $\hat{\lambda}$ (Eq. 6, clipped to $[0, 1]$) and $\hat{\eta}$ from the other fold; $\tilde{V}_k$ by Eq. (4)
7: $\hat{A}_k \leftarrow \tilde{V}_k - \tilde{V}_{k-1}$; DAPO update with token advantage $\hat{A}_k$
8: Add fork labels to the replay buffer; refit g_ϕ, h_ψ

B Implementation Details

Privileged view. The privileged prompt inserts the reference solution before the problem with a one-line instruction to read it and then solve the problem independently. Both views use the frozen π_{old} weights, and the plain-view log-probabilities and entropies are the ones DAPO already computes for the importance ratio.

Calibrator and residual model. g_ϕ is logistic regression on the five features of Section 3.3 plus an intercept, with an ℓ_2 penalty of 10^{-3}. h_ψ is linear regression on the same features predicting $(\hat{Y}_k - m_k)^2 - \hat{\sigma}_k^2$, an unbiased target for $(V_k - m_k)^2$, with predictions clipped at zero. Both are fit per fold on the other fold's labels and a replay buffer of the previous 20 batches. With $M = 1$ the denominator of $\hat{\sigma}_k^2$ vanishes, so for the $M = 1$ ablation h_ψ instead predicts the total squared residual $(\hat{Y}_k - m_k)^2$, whose conditional mean $\sigma_k^2 + (V_k - m_k)^2$ is the quantity in $p_k^\star$, and Equation (3) uses $u_k = \sqrt{h_\psi(z_k)}$. This changes only the fork probabilities, so Lemma 1 and Theorem 1 still hold.

Token-level credit. Every token of segment k receives $\hat{A}_k$, so the token-level credit of a response is $\sum_k |a_k| \hat{A}_k$, which generally differs from $R - b$; the identity in Equation (5) holds for segment credits only. We do not divide $\hat{A}_k$ by $|a_k|$: this per-token assignment is exactly the estimator $\hat{g}$ of Theorem 1, and length normalization would change it.

Diagnosis protocol. For Section 3.2 and Table 4, oracle continuations are sampled from the frozen checkpoint at temperature 1.0 without c. The control predictor uses $\log t_k$ and the plain-view entropy features. A segment is non-decisive when its oracle advantage satisfies $|A_k| < 0.02$.

Recurrent state. Forks restart from re-fed text. For the Gated DeltaNet layers of Qwen3.5 we store the recurrent state at each boundary during the original rollout and restore it when a fork starts, which adds 6% to rollout time. Attention layers use the inference engine's prefix cache.

Hyperparameters. AdamW with learning rate 1×10^{-6}, 128 prompts per step, 500 steps on Qwen3.5-9B (440 on Qwen3.5-4B and 400 on Olmo-3-7B, matched to their budgets of 3.4×10^9 and 3.1×10^9 tokens), DAPO clipping $(0.2, 0.28)$ with dynamic sampling disabled for all methods, token-level loss normalization, no KL penalty, temperature 1.0, maximum response length 8,192. ForkOPD: $N = 8$, $B = 1$, $M = 2$, $p_{\min} = 0.025$, $\hat{\lambda}$ clipped to $[0, 1]$, segments of 64 to 1,024 tokens.

C Budget Accounting

We count every generated token, including fork continuations and the tokens SRPO uses to self-localize errors. Scoring passes are forward-only, so they count toward wall-clock time and not toward the token budget. We measure generation per training step in units of one full response of mean length and divide it by 128, the number of prompts in a DAPO step. On Qwen3.5-9B this gives DAPO 16.0, ForkOPD 15.6, VinePPO 16.3, TreeRL 16.2 and SRPO 16.1. For a method that trains on 128 prompts per step this is the cost of one prompt. ForkOPD forks 1.06 times per response with $M = 2$, and a fork continuation averages 45% of a full response because it starts partway through, so one prompt costs $8\,(1+1.06\times 2\times 0.45) \approx 15.6$. VinePPO forks at every boundary with $M = 1$, so one of its prompts costs about $128 \times 16.3/36 \approx 58$ response-equivalents (7.2 per response, which has 12.2 interior boundaries on average); this is why it trains on only 36 prompts per step.

Over 500 steps, DAPO generates $500 \times 128 \times 16.0$ response-equivalents, so 3.9×10^9 tokens correspond to a mean response length of about 3.8K tokens, well below the 8,192-token cap. VinePPO generates $500 \times 36 \times 58 \approx 500 \times 128 \times 16.3$ response-equivalents, within 2% of DAPO at equal mean length.

D Baseline Details

We use the authors' code when available and otherwise follow the paper's description. Each method's own coefficients are tuned on 500 held-out DeepMath prompts with a 100-step budget on Qwen3.5-4B and reused for all models (Table 6).

Method	Coefficient	Grid (selected in bold)
RLSD	weight clip	$\{0.1, \mathbf{0.2}, 0.5\}$
HSD	correct peer	{random, **shortest**}
FlowBalance	(η_A, β_G)	$\{0.5, \mathbf{1}\} \times \{\mathbf{0.1}, 0.5\}$
OPSD	KL direction	{forward, **reverse**}
OC-GRPO	guidance length	$\{\mathbf{25}\%, 50\%\}$ of c
SRPO	suffix group size	$\{2, \mathbf{4}\}$
TreeRL	branches per node	$\{2, \mathbf{4}\}$
VinePPO	continuations	$\{\mathbf{1}, 2\}$

Table 6: Tuned baseline coefficients.

E Statistical Reporting

Every benchmark score is the mean over three seeds. Table 7 gives the standard deviation of the 9B five-benchmark average across seeds. A 95% bootstrap interval over problems and samples has a half-width of about 1.1 points for the five-benchmark average of any method. Comparisons use a paired permutation test at the problem level on the 590 pooled problems: ForkOPD versus HSD gives $p = 0.003$, and versus SRPO $p < 0.001$.

Method	Avg.	Method	Avg.
GRPO	24.12 ± 0.41	RLSD	25.86 ± 0.38
DAPO	25.78 ± 0.36	HSD	27.30 ± 0.35
VinePPO	25.06 ± 0.47	OC-GRPO	26.28 ± 0.42
TreeRL	25.20 ± 0.44	FlowBalance	26.52 ± 0.40
SRPO	26.90 ± 0.39	ForkOPD	30.06 ± 0.31
OPSD	19.48 ± 0.72		

Table 7: Five-benchmark average on Qwen3.5-9B, mean $\pm$ standard deviation over three seeds.